DMRC Helper: Complete Codebase Study Notes & Architecture Guide

Android • Kotlin • Jetpack Compose • Material 3

DMRC Helper: Complete Codebase Study Notes & Architecture Guide

\> **Project Name:** DMRC Helper (Smart Delhi Metro Companion)

\> **Platform:** Android (API 24 to 36)

\> **Language:** Kotlin

\> **UI Framework:** Jetpack Compose (Material 3)

\> **Architecture Pattern:** Single-Activity Architecture (`MainActivity` + Compose `NavHost`)

\> **Target Audience:** Commuters & Tourists traveling across the Delhi NCR Metro Network

\---

Table of Contents

1\. [Project Overview & Philosophy]\(#1-project-overview--philosophy)

2\. [High-Level Architecture & Tech Stack]\(#2-high-level-architecture--tech-stack)

3\. [Repository Directory & File Anatomy]\(#3-repository-directory--file-anatomy)

4\. [Data Layer & Asset Storage]\(#4-data-layer--asset-storage)

5\. [How the Files & Components Connect]\(#5-how-the-files--components-connect)

6\. [Deep-Dive: Screen-by-Screen Code Analysis]\(#6-deep-dive-screen-by-screen-code-analysis)

• [6.1 MainActivity & App Bootstrapping]\(#61-mainactivity--app-bootstrapping)

• [6.2 Smart Metro Dashboard (`DashboardHomeScreen`)]\(#62-smart-metro-dashboard-dashboardhomescreen)

• [6.3 Nearest Station Finder (`NearestStationScreen`)]\(#63-nearest-station-finder-neareststationscreen)

• [6.4 Route Planner (`RoutePlannerScreen`)]\(#64-route-planner-routeplannerscreen)

• [6.5 Interactive Metro Map (`MetroMapScreen`)]\(#65-interactive-metro-map-metromapscreen)

• [6.6 Metro Parking Directory (`AvailableParkingsScreen`)]\(#66-metro-parking-directory-availableparkingsscreen)

• [6.7 Ticket Booking Hub (`BookTicketsScreen`)]\(#67-ticket-booking-hub-bookticketscreen)

• [6.8 FAQ & Helpline (\FaqScreen\ & \FaqData.kt\)]\(#68-faq--helpline-faqscreen--faqdatakt)

• [6.9 Explore Delhi Tourist Guide (\ExploreDelhiScreen.kt\)]\(#69-explore-delhi-tourist-guide-exploredelhiscreenkt)

• [6.10 About App Screen (\AboutAppScreen\)]\(#610-about-app-screen-aboutappscreen)

7\. [End-to-End User Interaction Flows (Under the Hood)]\(#7-end-to-end-user-interaction-flows-under-the-hood)

8\. [Core Android & Jetpack Compose Concepts for Beginners]\(#8-core-android--jetpack-compose-concepts-for-beginners)

9\. [Important Observations & Edge Cases]\(#9-important-observations--edge-cases)

\---

1. Project Overview & Philosophy

DMRC Helper is an all-in-one transit utility application for the Delhi Metro Rail Corporation (DMRC) network. It solves common commuter pain points:

* Figuring out the nearest metro station from the user's current GPS location.

* Checking the road/walking path to that station directly on an interactive map.

* Calculating travel duration, distances, and fares between any two metro stations.

* Finding official parking lots, contractor names, and direct phone lines at stations.

* Directly opening official ticket booking avenues (DMRC QR portal and WhatsApp ticketing bot).

* Viewing an offline high-resolution metro map with pinch-to-zoom gestures.

* Getting quick answers to luggage allowances, smart card rules, and emergency helpline dialers.

* Exploring top monuments, street food lanes, and heritage markets located right outside major metro stations.

\---

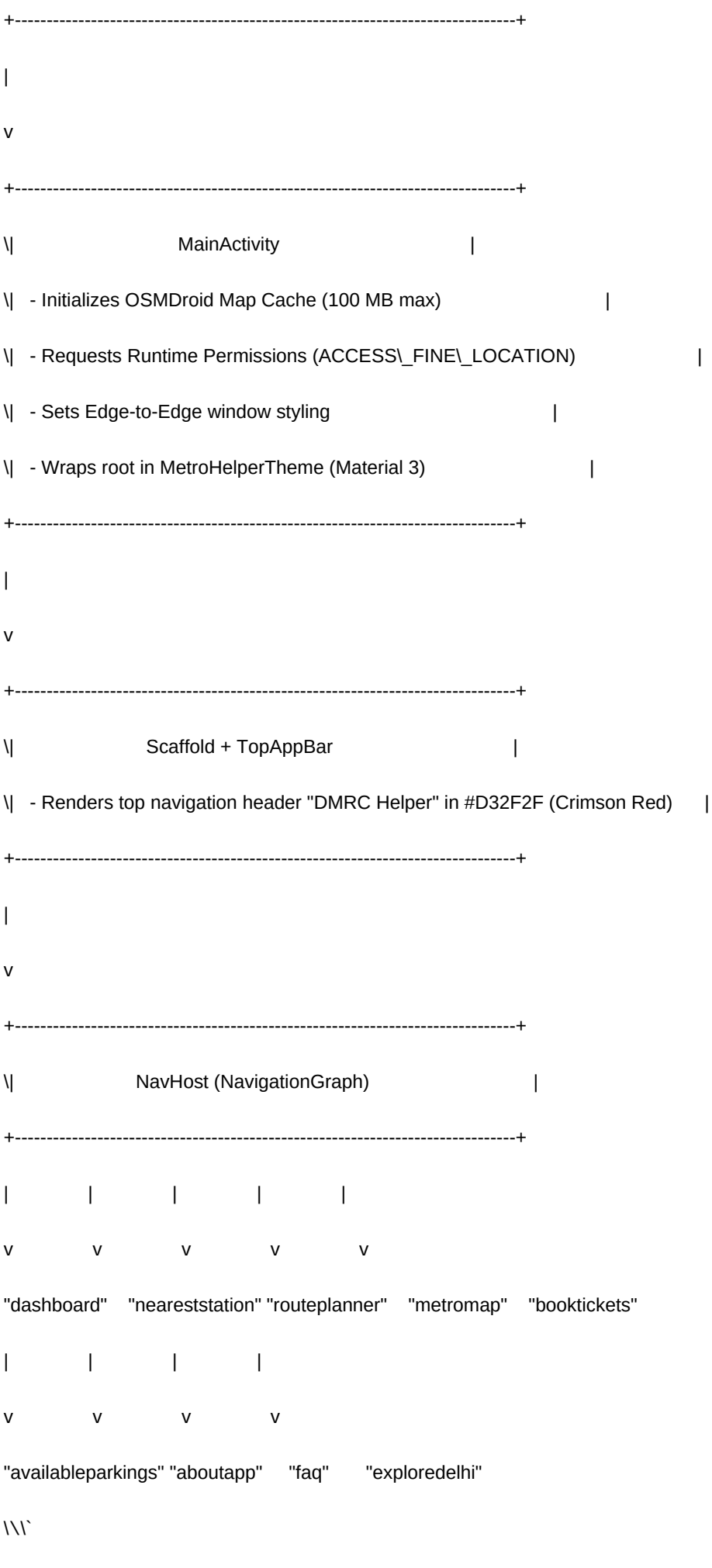
2. High-Level Architecture & Tech Stack

The application follows the **Single-Activity Architecture** recommended by Google. There is only **one Android Activity** (\MainActivity\), and every screen is a composable function managed by \NavigationGraph\ via \NavHostController\.

\\\`

+---+

\| Android OS |


+
|
v
+
\| MainActivity |
\| - Initializes OSMDroid Map Cache (100 MB max) |
\| - Requests Runtime Permissions (ACCESS_FINE_LOCATION) |
\| - Sets Edge-to-Edge window styling |
\| - Wraps root in MetroHelperTheme (Material 3) |
+
|
v
+
\| Scaffold + TopAppBar |
\| - Renders top navigation header "DMRC Helper" in #D32F2F (Crimson Red) |
+
|
v
+
\| NavHost (NavigationGraph) |
+
| | | | |
v v v v v
"dashboard" "nearestst
| | | |
v v v v
"availableparkings" "aboutapp" "faq" "exploredelhi"
\\\`

Key Libraries & Why They Were Selected

\| Library | Version | Purpose & Rationale |

\|---|---|---|

\| **Jetpack Compose BOM** | \`2024.10.00\` | Declarative UI toolkit. Eliminates XML layouts; UI is written entirely in Kotlin functions that automatically update with state. |

\| **Material 3 (\`androidx.compose.material3\`)** | Latest BOM | Modern styling tokens (Cards, TopAppBar, ExposedDropdownMenu, OutlinedTextField, FilterChips). |

\| **Navigation Compose** | \`2.8.3\` | Manages backstack navigation between composable destinations without launching separate Android activities. |

\| **OSMDroid (\`osmdroid-android\`)** | \`6.1.18\` | OpenStreetMap viewer for Android. Completely free and open-source; avoids Google Maps billing and API key requirements. |

\| **Google Play Services Location** | \`21.0.1\` | Battery-efficient \`FusedLocationProviderClient\` for fetching high-accuracy GPS coordinates. |

\| **OkHttp3** | \`4.12.0\` | High-performance HTTP client used to fetch GeoJSON routing data from the OSRM public REST API. |

\| **Kotlin Coroutines (\`kotlinx-coroutines-android\`)** | \`1.7.3\` | Asynchronous programming. Offloads file I/O and network requests to \`Dispatchers.IO\` to prevent UI freezes. |

\---

3. Repository Directory & File Anatomy

\\\`

d:\DMRC-Helper-master\\

■

■■■ gradle/

■ ■■■ libs.versions.toml # Dependency Version Catalog

■■■ app/

■ ■■■ build.gradle.kts # Module build configuration, SDK versions & libraries

■ ■■■ src/main/

■ ■ ■■■ AndroidManifest.xml # Permissions, application metadata & launch Activity

■ ■ ■■■ assets/

■ ■ ■ ■■■ dmrc_stations.json # Station names & GPS coordinates (Lat/Lon)

■ ■ ■ ■■■ delhi_metro_parking_clean.json # Parking directory (Stations, contractors, phones)

■ ■ ■■■ java/com/example/metrohelper/

■ ■ ■ ■■■ MainActivity.kt # Application entry point, NavHost & 6 feature screens

■ ■ ■ ■■■ FaqData.kt # Data classes & 10 static DMRC FAQ entries

■ ■ ■ ■■■ FaqScreen.kt # Searchable FAQ with expandable accordion cards & dialers

■ ■ ■ ■■■ ExploreDelhiScreen.kt # Tourist guide around major metro hubs (Master-Detail)

■ ■ ■ ■■■ ui/theme/

■ ■ ■ ■■■ Color.kt # Color definitions

■ ■ ■ ■■■ Theme.kt # Light/Dark and Android 12+ dynamic color schemes

■ ■ ■ ■■■ Type.kt # Material 3 typography definitions

■ ■ ■■■ res/

■ ■ ■■■ drawable/ # Raster map (mapview\.png) and tourist photos (*.jpg, *.png)

■ ■ ■■■ values/ # Strings, styles, and XML colors

■

■■■ DMRC-Helper/ # Nested folder containing an earlier project backup

\\\`

Detailed Purpose of Each Active File

1\. **\`gradle/libs.versions.toml\`**: The modern Gradle Version Catalog. Centralizes dependency versions (\`agp = "9.1.0"\`, \`kotlin = "2.2.10"\`, \`composeBom = "2024.09.00"\`).

2\. **\`app/build.gradle.kts\`**: Directs the Android build tool. Sets \`minSdk = 24\`, \`targetSdk = 36\`, enables Jetpack Compose compiler (\`compose = true\`), and links all dependencies.

3\. **\`AndroidManifest.xml\`**: Declares necessary permissions (\`ACCESS\_FINE\_LOCATION\`, \`ACCESS\_COARSE\_LOCATION\`, \`INTERNET\`), sets \`usesCleartextTraffic="true"\` to permit HTTP map tile calls, sets the OSMDroid user-agent, and declares \`MainActivity\` as the app launcher.

4\. **\`MainActivity.kt\`**: Contains the root Activity, OSMDroid initialization, permission request handlers, the Navigation Graph, and 6 full screens (\`DashboardHomeScreen\`, \`RoutePlannerScreen\`, \`NearestStationScreen\`, \`MetroMapScreen\`, \`BookTicketsScreen\`, \`AvailableParkingsScreen\`, \`AboutAppScreen\`).

5\. **\`FaqData.kt\`**: Defines the \`FaqItem\` data structure and the static \`FaqDataProvider\` containing categories and 10 verified DMRC guidelines.

6\. **\`FaqScreen.kt\`**: Implements category filtering chips, instant text search, smooth expandable accordion cards, and direct emergency dialer buttons.

7\. **\`ExploreDelhiScreen.kt\`**: Implements a master-detail destination guide for popular stations (Rajiv Chowk, Chandni Chowk, Central Secretariat, Hauz Khas, etc.) with ratings, photos, and transit tips.

8\. **\`ui/theme/Theme.kt\`**: Sets up \`MetroHelperTheme\`. Supports Android 12+ dynamic theming, dark mode, and light mode.

9\. **\`assets/dmrc\_stations.json\`**: Static offline dataset mapping every metro station to its latitude and longitude.

10\. **\`assets/delhi\_metro\_parking\_clean.json\`**: Static offline dataset mapping metro stations to parking contractors, contact phone numbers, lines, and notes.

\---

4. Data Layer & Asset Storage

The app minimizes reliance on remote servers by bundling vital transit data directly inside the APK's \`assets/\` directory:

1. \`dmrc\_stations.json\`

* **Total Entries:** \~250 stations

* **JSON Schema:**

\`\`\`json

{

"Station": "Rajiv Chowk",

"Line": "Blue Line",

"Latitude": 28.6328,

"Longitude": 77.2195

}

\`\`\`

* **Parsing Function:** \`loadMetroStations(context)\` in \`MainActivity.kt\`. It reads the asset as a text stream and populates a \`List\<MetroStation\>\`.

2. \`delhi\_metro\_parking\_clean.json\`

* **Total Entries:** \~150+ parking locations across all metro lines

* **JSON Schema:**

\\\`json

{

"id": 1,

"line": "Red Line",

"station": "Rithala",

"contractor": "M/s Shree Ram Parking",

"contact": "9810012345",

"additional_contacts": "",

"note": ""

}

\\\`

* **Parsing Function:** \`loadParkingStations(context)\` in \`MainActivity.kt\`. Runs asynchronously inside a Coroutine on \`Dispatchers.IO\` so the UI never drops frames while reading JSON.

\---

5. How the Files & Components Connect

\\\`

[User Launches App]

|

v

AndroidManifest.xml

|

v

MainActivity.kt

+----------------------+----------------------+

| | |

v v v

OSMDroid Cache Request GPS Permission MetroApp()

(100MB Disk Limit) (ACCESS_FINE_LOCATION) |

v

NavigationGraph (NavHost)

|

+----------+----------+----------+----------+

| | | | |

v v v v v

"dashboard" "nearestStation" "routeplanner" "metromap" "booktickets"

| | | | |

v | | | v

DashboardHomeScreen | | | BookTicketsScreen

| v v v |

| NearestStationScreen RoutePlannerScreen MetroMapScreen |

| | | | Intents to:

| +---------+---------+ v - DMRC QR Web

| | res/drawable/ - WhatsApp Bot

| v mapview\.png

| assets/dmrc_stations.json

| ^

| |

+----------+----------+

| | |

v v v

"availableparkings" "faq" "exploredelhi"

| | |

v v v

AvailableParkingsScreen FaqScreen ExploreDelhiScreen

| | |

v v v

assets/delhi_metro_... FaqData.kt res/drawable/*.jpg

| |

v v

Intent(ACTION_DIAL) Intent(ACTION_DIAL)

\\\`

\---

6. Deep-Dive: Screen-by-Screen Code Analysis

6.1 MainActivity & App Bootstrapping

* **File:** \`MainActivity.kt\`

* **What happens at startup:**

1. \`osmdroidConfig.load()\` loads saved tile preferences from Android \`SharedPreferences\`.

2. \`osmdroidConfig.userAgentValue = "DMRCHelper/1.0 (contact: ...)"\` ensures compliance with OpenStreetMap's fair-use tile policies so the tile server does not block requests.

3. \`tileFileSystemCacheMaxBytes = 100MB\` ensures downloaded map tiles remain stored locally offline.

4. Checks \`ActivityCompat.checkSelfPermission(Manifest.permission.ACCESS\_FINE\_LOCATION)\`. If missing, requests it immediately with request code \`100\`.

5. Calls \`enableEdgeToEdge()\` to extend UI behind the Android status and navigation bars.

6. Renders \`MetroApp()\`, which defines the \`Scaffold\` and \`NavigationGraph\`.

\---

6.2 Smart Metro Dashboard (\`DashboardHomeScreen\`)

* **Route:** \`"dashboard"\` (Start Destination)

* **Code Reference:** \`MainActivity.kt:L401-L544\`

* **Features & Logic:**

* **Dynamic Time Intelligence:** Uses \`Calendar.getInstance()\` to query current hour (\`HOUR\_OF\_DAY\`) and day (\`DAY\_OF\_WEEK\`):

* 08:00–10:00 -> "Rush Hour", Crowd: "Very High", Advice: "Use less crowded coach"

* 18:00–20:00 -> "Heavy Rush", Crowd: "High", Advice: "Metro is best option now"

* 11:00–16:00 -> "Perfect Time To Leave", Crowd: "Low"

* Sunday -> Advice: "Weekend travel is smooth today"

* **Commuter Savings Counter:** Reads \`trips\` (default 8) from \`SharedPreferences("metro\_stats")\` and calculates savings: \`savings = trips \* 25\`.

* **Utilities 2x4 Grid:** Renders 8 interactive cards navigating to every feature in the app.

\---

6.3 Nearest Station Finder (\`NearestStationScreen\`)

* **Route:** \`"neareststation"\`

* **Code Reference:** \`MainActivity.kt:L820-L1365\`

* **Features & Logic:**

* **GPS Prompt:** \`checkLocationEnabled()\` verifies that physical device location is toggled on. If off, displays the native Android prompt (\`ResolvableApiException\`).

* **Current Location:** Fetches live coordinates using \`FusedLocationProviderClient.getCurrentLocation(Priority.PRIORITY\_HIGH\_ACCURACY, null)\`.

* **Nearest Station Search:**

\\\`kotlin

fun findNearestStation(userLat: Double, userLon: Double, stations: List\<MetroStation>): MetroStation? {

return stations.minByOrNull {

val result = FloatArray(1)

Location.distanceBetween(userLat, userLon, it.lat, it.lon, result)

result[0] // Distance in meters

}

}

\\\`

Uses Android's native Haversine geodesic calculation across all stations in \`dmrc\_stations.json\`.

* **Routing API (OSRM):** Calls the public Open Source Routing Machine endpoint:

\https\://router.project-osrm.org/route/v1/driving/{userLon},{userLat};{stationLon},{stationLat}?overview=full&geometries=geojson\

Parses the GeoJSON coordinate array into OSMDroid \GeoPoint\ objects.

* **OSMDroid Map via \AndroidView\:**

* Injects OSMDroid \MapView\ into Compose.

* Places a \Marker\ at the user's location ("You are here").

* Places a \Marker\ at the nearest station.

* Adds two \Polyline\ overlays: an outer white line (width 14f) and an inner blue line (width 8f) to produce a clean bordered navigation path.

* **Floating Controls:** Floating Zoom In (\+\), Zoom Out (\-\), and Locate Me (\MyLocation\) buttons.

* **Bottom Summary Card:** Displays station name, distance in km (\%.1f km\), and walking ETA (\(distance * 12).toInt()\ minutes).

\---

6.4 Route Planner (\`RoutePlannerScreen\`)

* **Route:** \"routeplanner"\

* **Code Reference:** \MainActivity.kt:L248-L398\

* **Features & Logic:**

* Loads stations from \dmrc_stations.json\.

* Uses two Material 3 \ExposedDropdownMenuBox\ components for "From Station" and "To Station".

* Autocomplete search filters station names dynamically as the user types, taking the top 8 matches.

* When "Find Route" is tapped:

* \calculateDistance(startLat, startLon, endLat, endLon)\ returns aerial distance in km.

* Estimated time: \(distance * 3).toInt()\ minutes.

* Estimated fare: \(distance * 2).toInt()\ rupees.

* Formats result: \"Distance: 14.2 km\nEstimated Time: 42 mins\nFare: ■28"\.

\---

6.5 Interactive Metro Map (\`MetroMapScreen\`)

* **Route:** \"metromap"\

* **Code Reference:** `MainActivity.kt:L623-L672`

* **Features & Logic:**

* Displays offline raster map `R.drawable.mapview`.

* Uses Compose `pointerInput` with `detectTransformGestures { _, pan, zoom, _ -> ... }`.

* Zoom is clamped between `1.0f` (normal) and `4.0f` (4x magnification).

* Pan offsets are clamped to `500f * (newScale - 1)` so users cannot accidentally drag the map completely off the screen.

* Uses `graphicsLayer(scaleX, scaleY, translationX, translationY)` for hardware-accelerated rendering.

\---

6.6 Metro Parking Directory (\`AvailableParkingsScreen\`)

* **Route:** `"availableparkings"`

* **Code Reference:** `MainActivity.kt:L1562-L1746`

* **Features & Logic:**

* Asynchronously loads `delhi_metro_parking_clean.json` in a background coroutine (`Dispatchers.IO`).

* Real-time search bar filters both `station` and `line` fields simultaneously.

* Top bar of each card is color-coded using `lineColor(station.line)`:

* Red Line -> `#D32F2F`

* Yellow Line -> `#F9A825`

* Blue Line -> `#1565C0`

* Green Line -> `#2E7D32`

* Violet Line -> `#6A1B9A`

* Pink Line -> `#AD1457`

* Magenta Line -> `#880E4F`

* Displays contractor name, operational note, and phone number.

* Clicking any phone number triggers `Intent.ACTION_DIAL` with `tel:${station.contact}`, launching the device dialer.

* Empty state displays a "Not Available" badge with clear instructions when a search query yields no results.

\---

6.7 Ticket Booking Hub (\`BookTicketsScreen\`)

* **Route:** \"booktickets"\

* **Code Reference:** \MainActivity.kt:L1367-L1439\

* **Features & Logic:**

* Uses \openLinkSafely(context, url)\:

* **Option 1 (Official QR Web App):** Launches \https\://qrticket.dmrc.org/qrapp/\ in the device browser.

* **Option 2 (WhatsApp Chatbot):** Encodes the greeting \"Hi DMRC, I want to book a metro ticket."\ and opens \https\://wa.me/919650855800?text=...\.

* Wrapped in \try-catch\ to display a user-friendly Toast if no browser or WhatsApp is available.

\---

6.8 FAQ & Helpline (\`FaqScreen\` & \`FaqData.kt\`)

* **Route:** \"faq"\

* **Code Reference:** \FaqScreen.kt\ & \FaqData.kt\

* **Features & Logic:**

* **Data Model (\FaqItem\):** \id\, \question\, \answer\, \category\.

* **Categories:** "All", "Ticketing", "Timings", "Luggage & Rules", "Helpline", "App Features".

* **Search & Filter:** Dual filtering by active category chip and free-text search across both question and answer texts.

* **Expandable Accordion Cards:** Uses \animateContentSize()\ and \AnimatedVisibility\ with fade transitions for smooth opening/closing.

* **Emergency Helpline Card:** Bottom banner with direct calling buttons for:

* DMRC 24x7 Helpline (\155370\)

* Women Helpline (\01123415440\)

\---

6.9 Explore Delhi Tourist Guide (\`ExploreDelhiScreen.kt\`)

* **Route:** \"exploredelhi"\

* **Code Reference:** \ExploreDelhiScreen.kt\

* **Features & Logic:**

* **Curated Metro Hubs:** Rajiv Chowk, Chandni Chowk, Central Secretariat, Hauz Khas, INA, etc.

* **Master-Detail State Pattern:** Managed by \`var selectedStation by remember { mutableStateOf\<StationRecommendations?>(null) }\`.

* **Station List View (\`selectedStation == null\`):** Displays popular hubs with station name, connecting lines, and number of recommended spots.

* **Detail View (\`selectedStation != null\`):** Displays a back arrow button and full attraction cards:

* Photo from \`res/drawable/\` with rounded corners.

* Title, category badge, and Google star rating with review count.

* Specific transit instructions (e.g., \`"■ 4 min walk (Exit Gate 6)"\` or \`"■ 5 min E-Rickshaw (■20)"\`).

* Historical/cultural description.

\---

6.10 About App Screen (\`AboutAppScreen\`)

* **Route:** \`"aboutapp"\`

* **Code Reference:** \`MainActivity.kt:L1748-L1821\`

* **Features & Logic:**

* Displays app branding, mission statement, and version number (\`1.0\`).

* Credits the student development team: Harshit Verma, Devansh Jindal, Debanshu Mazumdar, Samarth Gupta, Abhay Pratab Singh, and Amit Shahare.

* Credits project guide: Mr. Munish Kumar.

* Explicitly states the academic disclaimer: **"This is an academic project and not officially affiliated with DMRC."**

\---

7. End-to-End User Interaction Flows (Under the Hood)

Flow 1: Finding Nearest Station & Viewing Route

\\\`

1\. User taps "Nearby" on Dashboard.

2\. navController.navigate("nearestStation") loads NearestStationScreen.

3\. checkLocationEnabled() calls Google Play Services SettingsClient.

■■■ If GPS is disabled, Android prompts user to turn it on.

4\. fusedLocationClient.getCurrentLocation() returns user Lat/Lon.

5\. findNearestStation() loops through dmrc_stations.json using Location.distanceBetween().

■■■ Matches closest station (e.g., "Barakhamba Road", 0.6 km).

6\. LaunchedEffect queries OSRM REST API via OkHttp on Dispatchers.IO.

■■■ Endpoint returns GeoJSON road coordinates between user and station.

7\. MapView updates overlays:

■■■ Marker 1: User position ("You are here")

■■■ Marker 2: Station position ("Barakhamba Road")

■■■ Dual Polyline: White outline + Blue driving path

8\. Camera animates to user location at zoom level 15.5.

9\. Bottom card displays distance (0.6 km) and walking ETA (\~7 mins).

\\\`

Flow 2: Searching and Calling Metro Parking

\\\`

1\. User taps "Parking" on Dashboard.

2\. LaunchedEffect(Unit) reads delhi_metro_parking_clean.json via Dispatchers.IO.

3\. User types "Yellow" or "Hauz Khas" in the search box.

4\. Filter logic instantly matches either station name or line name.

5\. LazyColumn renders matching ParkingStationCards with yellow header bars (#F9A825).

6\. User taps the contractor contact phone number.

7\. Intent(Intent.ACTION_DIAL, Uri.parse("tel:98100...")) is dispatched.

8\. Android dialer opens with the contractor's number pre-entered.

\\\`

\---

8. Core Android & Jetpack Compose Concepts for Beginners

\| Concept | Explanation | Real Example in this Codebase |

\|---|---|---|

\| **Declarative UI** | Instead of manually updating views (\`findViewById\`), you describe what the UI should look like for a given state. Compose redraws automatically when state changes. | When \`searchQuery\` changes in \`AvailableParkingsScreen\`, the station list updates instantly. |

\| **\`remember { mutableStateOf() }\`** | Tells Compose to allocate and hold a variable across recompositions. | \`var expandedFrom by remember { mutableStateOf(false) }\` in \`RoutePlannerScreen\`. |

\| **\`LaunchedEffect\`** | A composable that safely launches a coroutine when entering the composition or when a key parameter changes. | Used in \`NearestStationScreen\` to trigger GPS queries and OSRM network requests. |

\| **\`Dispatchers.IO\`** | A Coroutine dispatcher optimized for disk reading and network requests off the UI thread. | Used in \`getRoute()\` for OkHttp and \`loadParkingStations()\` for JSON reading. |

\| **Android Intents** | Inter-application communication mechanism. | Used in \`BookTicketsScreen\` (\`ACTION_VIEW\`) to open browser/WhatsApp, and in \`ParkingStationCard\` (\`ACTION_DIAL\`) to open the phone app. |

\| **\`AndroidView\`** | A special Compose wrapper allowing traditional Android View widgets to live inside Compose. | Used to embed OSMDroid's native \`MapView\` inside \`NearestStationScreen\`. |

\| **Master-Detail Pattern** | A UI design pattern where a master list leads to a detailed item view on the same screen. | Used in \`ExploreDelhiScreen\` to switch between the station directory and attraction cards. |

\---

9. Important Observations & Edge Cases

1\. **Nested \`DMRC-Helper\` Folder:**

The workspace contains a subfolder named \`DMRC-Helper\` containing an earlier version of the project. The active, up-to-date application code is in the root \`app/src/main/\` folder.

2\. **Offline-First Architecture:**

Station coordinates, parking contractors, FAQs, Explore Delhi attractions, and the metro map raster are 100% offline. Only live OSM map tiles and OSRM turn-by-turn road paths require an internet connection.

3\. **Linear Route Heuristic:**

\`RoutePlannerScreen\` uses a linear formula (\`time = distance \* 3 min\`, \`fare = distance \* 2\`) rather than a heavy transit graph-search algorithm. This guarantees instant, offline responses with zero server costs.

4\. **Tile Cache Optimization:**

\`MainActivity.kt\` explicitly configures OSMDroid's disk cache to 100MB, allowing previously viewed map areas to render even if mobile data drops while underground.

\---

**End of Study Notes. Prepared for pair-programming and code mastery.\